homework3

2026-09-12

Homework 3

Question 1

I created the `explain-to-me` skill and tested it on Question 2. The test identified the goal as comparing how ridge stabilizes nearly collinear predictors without giving the calculations or final answer. The complete `SKILL.md` is included at the end.

Question 2

a) The first two standardized predictors have correlation

$$\text{cor}(X_1, X_2) = 0.9983,$$

with largest singular value 14.1730 and smallest singular value 0.4061. Since X_1 and X_2 are almost the same, their sum is well identified while their difference lies close to a weak singular direction, making $\hat{\beta}_1 - \hat{\beta}_2$ much more variable.

b)

λ	$\hat{\beta}_1$ mean (SD)	$\hat{\beta}_2$ mean (SD)	Sum mean (SD)	Difference mean (SD)
0	1.290 (1.795)	1.701 (1.798)	2.992 (0.105)	-0.411 (3.592)
0.02	1.465 (0.145)	1.497 (0.148)	2.961 (0.104)	-0.032 (0.274)
0.20	1.355 (0.049)	1.359 (0.050)	2.715 (0.095)	-0.004 (0.029)

λ	Average training MSE	Average test MSE
0	0.949	1.038
0.02	0.960	1.029
0.20	1.064	1.125

c) Ridge adds some bias but greatly reduces the variance of the separate coefficients, especially the unstable difference. The fitted values change much less because prediction depends mainly on the stable combined direction $\beta_1 + \beta_2$ rather than on how that effect is split between two nearly collinear predictors.

```
import numpy as np

rng = np.random.default_rng(43231)
n, p, R = 100, 4, 200
```

```
L = rng.normal(size=n)
U1 = rng.normal(size=n)
U2 = rng.normal(size=n)
X = np.column_stack([
    L + 0.04 * U1,
    L + 0.04 * U2,
    rng.normal(size=n),
    rng.normal(size=n),
])
X = X - X.mean(axis=0)
X = X / np.sqrt(np.mean(X**2, axis=0))

beta = np.array([1.5, 1.5, 1.0, 0.0])
mu = X @ beta
lambdas = np.array([0.0, 0.02, 0.2])
B = np.empty((R, p, len(lambdas)))
train = np.empty((R, len(lambdas)))
test = np.empty((R, len(lambdas)))

for r in range(R):
    y = mu + rng.normal(size=n)
    y_test = mu + rng.normal(size=n)
    yc = y - y.mean()
    for j, lam in enumerate(lambdas):
        B[r, :, j] = np.linalg.solve(
            X.T @ X + n * lam * np.eye(p), X.T @ yc
        )
        y_hat = y.mean() + X @ B[r, :, j]
        train[r, j] = np.mean((y - y_hat)**2)
        test[r, j] = np.mean((y_test - y_hat)**2)

s = np.linalg.svd(X, compute_uv=False)
print(np.corrcoef(X[:, 0], X[:, 1])[0, 1], s.max(), s.min())
for j, lam in enumerate(lambdas):
    vals = [B[:, 0, j], B[:, 1, j],
            B[:, 0, j] + B[:, 1, j],
            B[:, 0, j] - B[:, 1, j]]
    print(lam, [(z.mean(), z.std(ddof=1)) for z in vals],
          train[:, j].mean(), test[:, j].mean())
```

Question 3

a)

$$\nabla L_\lambda(\beta) = -\frac{1}{n}\mathbf{X}^\mathsf{T}(\tilde{\mathbf{y}} - \mathbf{X}\beta) + \lambda\beta.$$

Setting the gradient equal to zero gives

$$(\mathbf{X}^\mathsf{T}\mathbf{X} + n\lambda\mathbf{I}_p)\hat{\beta}_\lambda = \mathbf{X}^\mathsf{T}\tilde{\mathbf{y}}.$$

For any nonzero $\mathbf{a}$,

$$\mathbf{a}^\mathsf{T}(\mathbf{X}^\mathsf{T}\mathbf{X} + n\lambda\mathbf{I}_p)\mathbf{a} = \|\mathbf{X}\mathbf{a}\|^2 + n\lambda\|\mathbf{a}\|^2 > 0.$$

Thus, when $\lambda > 0$ the matrix is positive definite and invertible, so the ridge solution is unique even if $\mathbf{X}$ is rank deficient or $p > n$.

b)

Using $\mathbf{X} = \mathbf{U}_r\mathbf{D}_r\mathbf{V}_r^\top$,

$$\hat{\beta}_\lambda = \mathbf{V}_r(\mathbf{D}_r^2 + n\lambda\mathbf{I}_r)^{-1}\mathbf{D}_r\mathbf{U}_r^\top\tilde{\mathbf{y}},$$

so

$$\mathbf{X}\hat{\beta}_\lambda = \sum_{j=1}^{r} \frac{d_j^2}{d_j^2 + n\lambda}\mathbf{u}_j\mathbf{u}_j^\top\tilde{\mathbf{y}}.$$

With $n\lambda = 1$,

$$\rho_1 = \frac{100}{101} = 0.9901, \qquad \rho_2 = \frac{4}{5} = 0.8000, \qquad \rho_3 = \frac{0.04}{1.04} = 0.0385.$$

$$\mathrm{df}_{\mathrm{eff}} = 1 + 0.9901 + 0.8000 + 0.0385 = 2.8286.$$

c) The $d_3 = 0.2$ direction receives the strongest shrinkage because it is the least informed direction and therefore has the highest potential variance. This reduces prediction variance, but it can create substantial bias if the true mean response has a large component in that same direction.

Question 4

a)

$$\kappa(\mathbf{A}_0) = \frac{9}{0.01} = 900,$$

and ridge changes the eigenvalues to 9.09 and 0.10, so

$$\kappa(\mathbf{A}_\lambda) = \frac{9.09}{0.10} = 90.9.$$

The ridge objective is much less elongated because the weak-curvature direction is strengthened. This improves conditioning and makes the optimization geometry more balanced.

b) Since the ridge objective is quadratic,

$$\nabla L_\lambda(\beta^{(k)}) = \mathbf{A}_\lambda\mathbf{e}^{(k)},$$

so

$$\mathbf{e}^{(k+1)} = (\mathbf{I} - \eta\mathbf{A}_\lambda)\mathbf{e}^{(k)}.$$

Therefore, an eigendirection with unpenalized eigenvalue a is multiplied by

$$1 - \eta(a + \lambda).$$

Every direction contracts when

$$0 < \eta < \frac{2}{9.09} \approx 0.2200.$$

For $\eta = 1/M$,

$$1 - \frac{9.09}{9.09} = 0, \qquad 1 - \frac{0.10}{9.09} = 0.9890.$$

The strong direction converges immediately, while the weak direction converges slowly. For $\eta = 2.1/M$, the strong-direction factor is -1.1, whose magnitude exceeds one, so the method does not converge.

c) A larger λ may make optimization easier, but it also changes the statistical estimator by increasing shrinkage bias. Therefore, λ should be selected using prediction criteria such as cross-validation or GCV, not by whichever value makes gradient descent fastest.

Question 5

a)

$$\lambda_{\text{min}} = 0.01995, \qquad \lambda_{\text{1se}} = 1.58489.$$

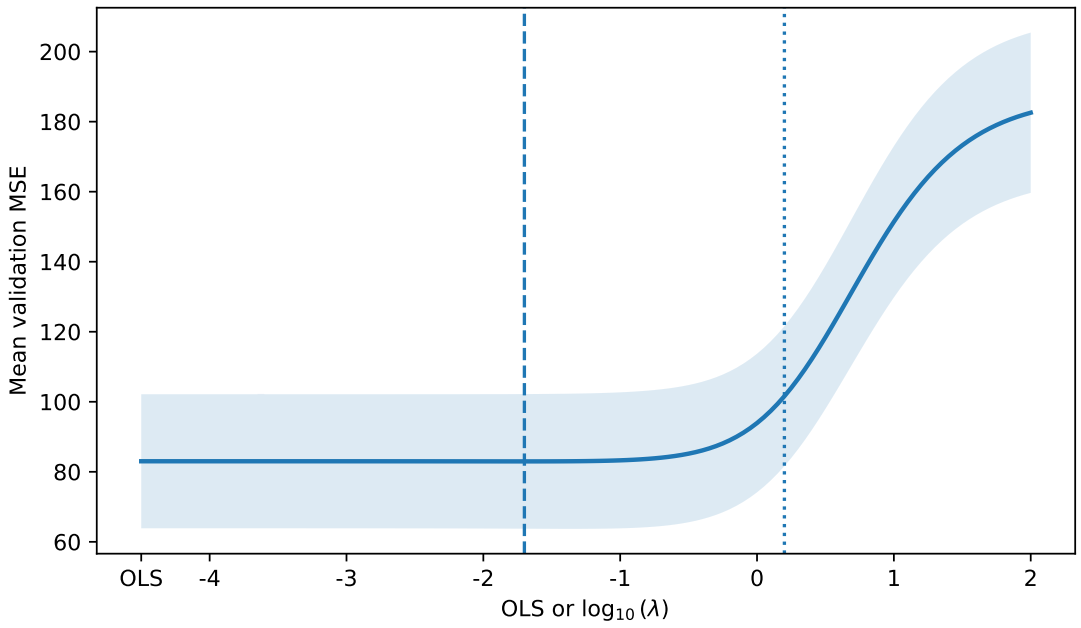

Figure 1: Ten-fold cross-validation mean MSE.

b)

$$\lambda_{\text{GCV}} = 0.02818.$$

c)

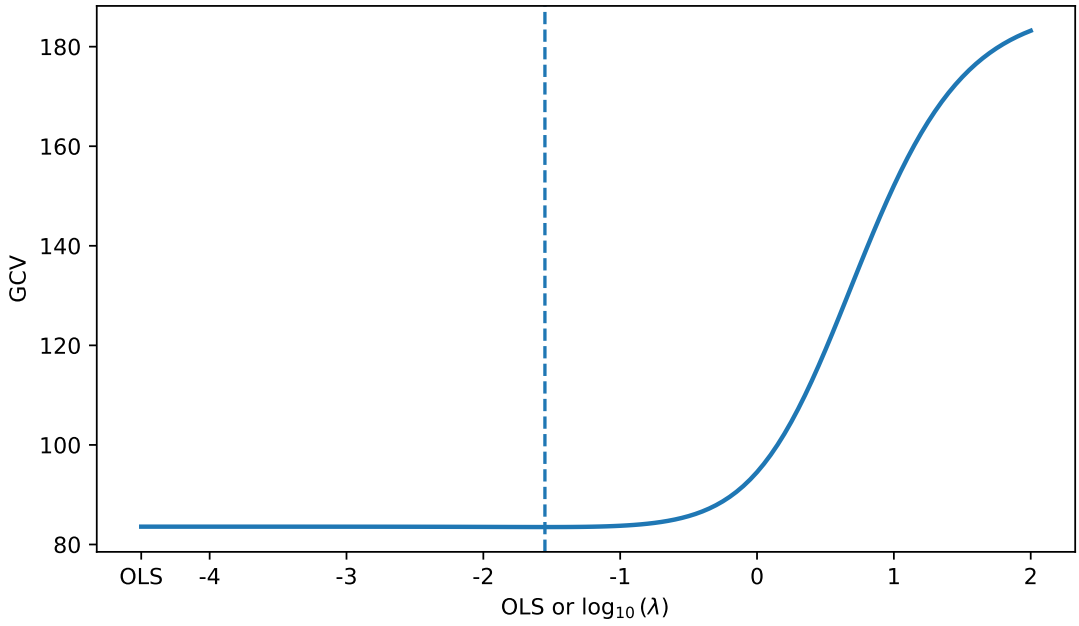

Figure 2: GCV over the ridge penalty grid.

Rule	λ	Effective df	Training MSE	Test MSE
OLS	0	7.000	80.094	66.322
CV minimum	0.01995	6.773	80.135	66.000
One SE	1.58489	3.033	100.374	81.200
GCV	0.02818	6.689	80.171	65.903

The CV-minimum and GCV fits use mild shrinkage and have very similar performance, while the one-SE rule uses much stronger shrinkage and fewer effective degrees of freedom. The smallest observed test MSE cannot be used to choose the model after the fact because that would turn the final test set into another validation set and bias the reported performance downward.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

realestate = pd.read_csv("data/realestate.csv")
fold_info = pd.read_csv("data/realestate-split-folds.csv")
data = fold_info.merge(realestate, on="row_id", validate="one_to_one")
vars = ["date", "age", "distance", "stores", "latitude", "longitude"]
X_all = data[vars].to_numpy(float)
y_all = data["price"].to_numpy(float)
is_train = data["split"].eq("train").to_numpy()
Xtr, Xte = X_all[is_train], X_all[~is_train]
ytr, yte = y_all[is_train], y_all[~is_train]
folds = data.loc[is_train, "cv_fold"].to_numpy(int)
lam_grid = np.r_[0.0, 10 ** np.linspace(-4, 2, 121)]

fold_mse = np.empty((10, len(lam_grid)))
for k, fold in enumerate(np.sort(np.unique(folds))):
```

```
    val = folds == fold
    Xfit, Xval = Xtr[~val], Xtr[val]
    yfit, yval = ytr[~val], ytr[val]
    xb = Xfit.mean(0)
    s = np.sqrt(np.mean((Xfit - xb)**2, axis=0))
    X = (Xfit - xb) / s
    Xv = (Xval - xb) / s
    yb = yfit.mean()
    yc = yfit - yb
    G, r = X.T @ X, X.T @ yc
    for j, lam in enumerate(lam_grid):
        b = np.linalg.solve(G + len(yfit) * lam * np.eye(6), r)
        fold_mse[k, j] = np.mean((yval - yb - Xv @ b)**2)

cv_mean = fold_mse.mean(0)
cv_se = fold_mse.std(0, ddof=1) / np.sqrt(10)
i_min = np.argmin(cv_mean)
lam_min = lam_grid[i_min]
cutoff = cv_mean[i_min] + cv_se[i_min]
lam_1se = lam_grid[cv_mean <= cutoff].max()

xb = Xtr.mean(0)
s = np.sqrt(np.mean((Xtr - xb)**2, axis=0))
X = (Xtr - xb) / s
yb = ytr.mean()
yc = ytr - yb
G, r = X.T @ X, X.T @ yc
d = np.linalg.svd(X, compute_uv=False)
gcv = np.empty(len(lam_grid))
df = np.empty(len(lam_grid))
for j, lam in enumerate(lam_grid):
    b = np.linalg.solve(G + len(ytr) * lam * np.eye(6), r)
    yhat = yb + X @ b
    df[j] = 1 + np.sum(d**2 / (d**2 + len(ytr) * lam))
    mse = np.mean((ytr - yhat)**2)
    gcv[j] = mse / (1 - df[j] / len(ytr))**2
lam_gcv = lam_grid[np.argmin(gcv)]

x = np.full(len(lam_grid), -4.5)
x[lam_grid > 0] = np.log10(lam_grid[lam_grid > 0])
plt.plot(x, cv_mean)
plt.fill_between(x, cv_mean-cv_se, cv_mean+cv_se, alpha=.15)
plt.show()
plt.plot(x, gcv)
plt.show()

def fit_ridge(X, y, lam, Xnew):
    xb = X.mean(0)
    s = np.sqrt(np.mean((X - xb)**2, axis=0))
    Z = (X - xb) / s
    yb = y.mean()
    b = np.linalg.solve(Z.T @ Z + len(y) * lam * np.eye(6),
                        Z.T @ (y - yb))
    braw = b / s
```

```
    a = yb - xb @ braw
    d = np.linalg.svd(Z, compute_uv=False)
    df = 1 + np.sum(d**2 / (d**2 + len(y) * lam))
    return a + X @ braw, a + Xnew @ braw, df

for name, lam in [("OLS", 0), ("CV minimum", lam_min),
                  ("One SE", lam_1se), ("GCV", lam_gcv)]:
    yh, yp, df = fit_ridge(Xtr, ytr, lam, Xte)
    print(name, lam, df,
          np.mean((ytr-yh)**2), np.mean((yte-yp)**2))
```

explain-to-me skill

```
---
name: explain-to-me
description: Help me understand a STAT 432 homework question without solving it.
---

# Intent

Help me understand the question without giving me the solution.

# Instructions

- Read the entire question before responding.
- Explain the main goal and what I am expected to submit in clear language.
- Keep the notation used in the question.
- Point me to the relevant Week 3 ridge-regression lecture when useful.
- Suggest one reasonable first step.
- Do not complete calculations, derivations, solution code, or give the final answer.
```